

Семинар #2: Функции и типы. Домашнее задание.

Для сдачи ДЗ вам нужно создать репозиторий на GitHub (если он ещё не создан) под названием `cpp-mipt-homework`. Структура репозитория должна иметь вид:

```
├── ...
├── seminar2_function_and_type/
│   ├── 01.c
│   ├── 02.c
│   └── ...
└── ...
```

Для каждой задачи, если не сказано иное, нужно создать один файл решения с расширением `.c`.

Задача 1. Куб

Напишите функцию `cube`, которая будет принимать на вход одно целое число и возвращать куб этого числа. Вызовите эту функцию в функции `main` следующим образом:

```
#include <stdio.h>

// Тут вам нужно написать
// функцию cube

int main()
{
    printf("%i\n", cube(5));
}
```

После того как вы напишете функцию `cube`, скомпилируйте данную программу и запустите она должна напечатать на экран число 125.

Задача 2. Четность

Напишите функцию `is_even`, которая будет принимать на вход одно целое число и возвращать 1, если это число чётное и 0, если число нечётное. Вызовите эту функцию в функции `main` следующим образом:

```
#include <stdio.h>

// Тут вам нужно написать
// функцию is_even

int main()
{
    printf("%i\n", is_even(90));
    printf("%i\n", is_even(91));
}
```

После того как вы напишете функцию `is_even`, скомпилируйте данную программу и запустите она должна напечатать на экран:

```
1
0
```

Задача 3. Печать чётных чисел

Напишите функцию `print_even`, которая будет принимать на вход два целых числа `a` и `b` и печатать на экран все чётные числа, которые находятся на отрезке `[a, b]`.

вход функции <code>print_even</code>	печать на экран
2 15	2 4 6 8 10 12 14
1 15	2 4 6 8 10 12 14
-7 3	-6 -4 -2 0 2

Задача 4. Сумма цифр числа

- Напишите функцию `sum_of_digits`, которая будет принимать на вход целое число и возвращать сумму всех цифр числа (в десятичной записи). Предполагайте, что на вход функции всегда будут приходиться неотрицательные числа. Реализуйте эту функцию с помощью цикла.

вход функции <code>sum_of_digits</code>	выход функции <code>sum_of_digits</code>
123	6
55955	29
4	4
0	0

- Напишите функцию `sum_of_digits_rec`, которая будет принимать на вход целое число и возвращать сумму всех цифр числа (в десятичной записи). Предполагайте, что на вход функции всегда будут приходиться неотрицательные числа. Реализуйте эту функцию с помощью рекурсии. Пользоваться уже написанной с помощью цикла функцией `sum_of_digits` в этой функции нельзя.

Задача 5. Бинарное представление числа

Напишите функцию `print_binary`, которая будет принимать на вход целое число и печатать на экран бинарное представление этого числа. Предполагайте, что на вход функции всегда будут приходиться неотрицательные числа. Реализуйте эту функцию с помощью рекурсии.

вход функции <code>print_binary</code>	печать на экран
6	110
128	10000000
4823564	10010011001101000001100
0	0

Задача 6. Числа трибоначчи

Числа трибоначчи - это последовательность чисел, задаваемые следующим образом:

```
trib(0) = 0;
trib(1) = 0;
trib(2) = 1;
trib(n) = trib(n - 3) + trib(n - 2) + trib(n - 1);
```

Напишите функцию, `trib`, которая будет принимать на вход целое число `n` и будет возвращать `n`-е число трибоначчи. Убедитесь, что функция работает быстро при вычислении 38-го числа трибоначчи.

вход функции <code>trib</code>	выход функции <code>trib</code>
1	0
5	4
20	35890
35	334745777
38	2082876103

Задача 7. Количество чётных

Напишите функцию `count_even`, которая будет принимать на вход массив целых чисел и количество элементов этого массива. Эта функция должна возвращать количество чётных чисел в этом массиве. Протестируйте эту функцию в функции `main`.

вход функции <code>count_even</code>	выход функции <code>count_even</code>
<code>array: 1 2 3 4 5</code> <code>size: 5</code>	2
<code>array: 10 20 30 40</code> <code>size: 4</code>	4
<code>array: 10 1</code> <code>size: 2</code>	1

Задача 8. Обратный массив

Напишите функцию `reverse`, которая будет принимать на вход массив целых чисел и количество элементов этого массива. Эта функция должна переворачивать массив задом наперёд. Протестируйте эту функцию в функции `main`.

вход функции <code>reverse</code>	массив после выполнения <code>reverse</code>
<code>array: 10 20 30 40 50</code> <code>size: 5</code>	50 40 30 20 10
<code>array: 60 20 80 10</code> <code>size: 4</code>	10 80 20 60

Задача 9. Необычная рекурсия

Алиса и Боб играют в игру. Сначала Алиса получает некоторое нечётное число. Алиса умножает это число на 3 и прибавляет к результату умножения единицу. Получившееся число Алиса печатает на экран и передаёт Бобу. Боб получает чётное число и делит это число на 2 пока число не станет нечётным. После каждого деления на 2, Боб печатает число на экран. Как только Боб получит нечётное число, отличное от 1, он передаёт её Алисе. Но если Боб получит число 1, то программа заканчивает выполнение.

Например, если мы передадим на вход программе число 13, то она должна напечатать следующее:

```
Alice: 40
Bob: 20
Bob: 10
Bob: 5
Alice: 16
Bob: 8
Bob: 4
Bob: 2
Bob: 1
```

Эту программу очень просто написать с помощью обычного цикла. Но ваша задача заключается в том, чтобы написать её с помощью рекурсии, причём поведение Алисы и Боба должны описываться двумя разными функциями. Напишите следующие функции:

- `void alice(int n)`
- `void bob(int n)`

Эти функции должны изменять проходящее на вход число, печатать его на экран и передавать новое число другой функции. То есть функция `alice` должна вызывать функцию `bob`, а функция `bob` должна вызывать функцию `alice` (но только если новое нечётное число будет отлично от единицы).

Задача 10. Присвоение матриц

Написать функцию `void assign(float A[MAX][MAX], float B[MAX][MAX], int n)`, которая присваивает элементам матрицы A соответствующие элементы матрицы B ($A = B$). Где MAX это `#define`-константа.

Задача 11. Умножение матриц

Написать функцию `void multiply(float A[MAX][MAX], float B[MAX][MAX], float C[MAX][MAX], int n)`, которая перемножает матрицы `A` и `B` размера n на n (строка на столбец), а результат записывает в матрицу `C`. Формула: $C_{ij} = \sum_k A_{ik} \cdot B_{kj}$.

Проверьте ваш код на следующих тестах:

$$\begin{pmatrix} 7 & 7 & 2 \\ 1 & 8 & 3 \\ 2 & 1 & 6 \end{pmatrix} \cdot \begin{pmatrix} 5 & 2 & 9 \\ -4 & 2 & 11 \\ 7 & 1 & -5 \end{pmatrix} = \begin{pmatrix} 21 & 30 & 130 \\ -6 & 21 & 82 \\ 48 & 12 & -1 \end{pmatrix}$$

$$\begin{pmatrix} 5 & 2 & 9 \\ -4 & 2 & 11 \\ 7 & 1 & -5 \end{pmatrix} \cdot \begin{pmatrix} 7 & 7 & 2 \\ 1 & 8 & 3 \\ 2 & 1 & 6 \end{pmatrix} = \begin{pmatrix} 55 & 60 & 70 \\ -4 & -1 & 64 \\ 40 & 52 & -13 \end{pmatrix}$$

$$\begin{pmatrix} 7 & 7 & 2 \\ 1 & 8 & 3 \\ 2 & 1 & 6 \end{pmatrix} \cdot \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 7 & 7 \\ 3 & 8 & 1 \\ 6 & 1 & 2 \end{pmatrix}$$

В файлах `matA.txt` и `matB.txt` лежат матрицы 10 на 10. Считайте эти матрицы с помощью метода перенаправления потока из файла `combinedAB.txt`, перемножьте (`A` на `B`) и запишите результат в другой файл. В результате должно получиться:

$$\begin{pmatrix} 259 & -15 & 237 & 257 & 231 & 67 & 237 & -64 & 152 & 363 \\ 555 & 233 & 539 & 188 & 356 & 325 & 423 & -47 & 123 & 387 \\ 497 & 512 & 572 & 95 & 619 & 155 & 414 & 207 & 203 & 217 \\ 455 & 280 & 675 & 354 & 664 & 346 & 483 & 177 & 168 & 404 \\ 264 & 182 & 272 & 290 & 474 & -33 & 234 & 99 & 379 & 156 \\ 272 & 180 & 469 & 286 & 326 & 282 & 325 & 215 & 195 & 231 \\ 421 & 363 & 475 & 506 & 359 & 481 & 468 & 101 & 325 & 328 \\ 384 & 218 & 567 & 395 & 475 & 488 & 361 & 168 & 291 & 298 \\ 387 & 297 & 480 & 170 & 318 & 423 & 483 & 10 & -17 & 406 \\ 193 & 241 & 486 & 38 & 403 & 146 & 286 & 326 & 212 & 172 \end{pmatrix}$$

Задача 12. Матрица в степени

Написать функцию `void power(int A[MAX][MAX], int C[MAX][MAX], int n, int k)`, которая вычисляет A^k , т.е. возводит матрицу `A` размера n на n в k -ю степень, а результат записывает в матрицу `C`. Используйте функции `multiply` и `assign`. Псевдокод простейшей реализации такой функции:

```
float B[MAX][MAX]

B = A    // конечно нельзя приравнять массивы, но можно
C = A    // использовать вашу функцию assign

for ( k - 1 раз )
{
    C = A * B
    B = C
}
```

Проверьте ваш код на следующих тестах:

$$\begin{pmatrix} 7 & 7 & 2 \\ 1 & 8 & 3 \\ 2 & 1 & 6 \end{pmatrix}^4 = \begin{pmatrix} 7116 & 15654 & 9549 \\ 4002 & 8955 & 6135 \\ 3369 & 6165 & 4350 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}^{70} = \begin{pmatrix} 145547525 & 109870576 & 82938844 \\ 192809420 & 145547525 & 109870576 \\ 109870576 & 82938844 & 62608681 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}^{1000} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

Подумайте, как можно возвести матрицу в k -ю степень более эффективно. Рассмотрите случай, когда k является степенью двойки и когда не является.

Типы данных

Основные типы и их стандартные размеры на 64-х битных системах

тип	размер (байт)	диапазон значений ($2^{\#bits}$)	спецификатор
char	1	от -128 до 127	%hhi
short	2	от -32768 до 32767	%hi
int	4	примерно от -2-х миллиардов до 2-х миллиардов	%i
long	4	примерно от -2-х миллиардов до 2-х миллиардов	%li
long long	8	примерно от -10^{19} до 10^{19}	%lli
unsigned char	1	от 0 до 255	%hhu
unsigned short	2	от 0 до 65535	%hu
unsigned int	4	от 0 до $2^{32} \approx 4 * 10^9$	%u
unsigned long	4	от 0 до $2^{32} \approx 4 * 10^9$	%lu
unsigned long long	8	от 0 до $2^{64} \approx 2 * 10^{19}$	%llu
size_t	8	от 0 до $2^{64} \approx 2 * 10^{19}$	%zu

тип	размер (байт)	значимые цифры	диапазон экспоненты	спецификатор
float	4	6	от -38 до 38	%f
double	8	15	от -308 до 308	%lf
long double	от 8 до 16	≥ 15	не хуже чем у double	%Lf
печать только 3-х чисел после запятой	-	-	-	%.3f
печать без нулей на конце	-	-	-	%g
печать в научной записи	-	-	-	%e

тип	размер (байт)	спецификатор
указатель	8	%p

Задача 13. Факториал

Для вычисления факториала была написана следующая простая программа.

```
#include <stdio.h>
int fact(int n)
{
    int result = 1;
    for (int i = 1; i <= n; ++i)
        result *= i;
    return result;
}

int main()
{
    int k;
    scanf("%i", &k);
    printf("%i\n", fact(k));
}
```

Однако, выяснилось, что эта программа правильно работает только для k от 0 до 12. При больших k программа выдаёт неверный ответ. Почему это происходит? Немного измените программу, чтобы она работала для k до 20 включительно.

ВХОД	ВЫХОД
5	120
13	6227020800
17	355687428096000
20	2432902008176640000

Задача 14. Размещения

В комбинаторике размещением (из n по k) A_n^k называется упорядоченный набор из k различных элементов из некоторого множества различных n элементов. Размещения вычисляются следующим образом: $A_n^k = \frac{n!}{(n-k)!}$. Напишите программу, которая будет вычислять размещения при условии, что $A_n^k < 2^{64}$. Проверьте вашу функцию на следующих значениях:

ВХОД	ВЫХОД
5 2	20
20 10	670442572800
30 12	41430393164160000
60 11	13679492361575040000

Задача 15. Часть года

Напишите функцию `float yearfrac(int year, int day)`, которая принимает номер года `year` и номер дня с начала года `day` и возвращает прошедшую долю года. В этой задаче считайте, что високосный год, это год, чей номер делится на 4 (хотя это не совсем так).

year	day	yearfrac(year, day)
2019	300	0.82192
2019	100	0.27397
2020	100	0.27322

Задача 16. Вычисление π

Известно, что число π можно вычислить с помощью следующего ряда:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots = \sum_{i=1}^{\infty} \frac{(-1)^{i+1}}{2i-1}$$

Используйте эту формулу, чтобы вычислить приблизительно число π . На вход должно подаваться целое число n - число членов суммируемой последовательности, а вам нужно вычислить приближённое значение:

$$\pi \approx 4 \cdot \sum_{i=1}^n \frac{(-1)^{i+1}}{2i-1}$$

Задача 17. Два круга

Напишите программу, которая проверяет пересекаются ли 2 круга. Программа должна принимать на вход координаты центров кругов и их радиусы в следующем порядке:

```
x1 y1 r1
x2 y2 r2
```

и печатать следующее:

- `Do not intersect` – если окружности не пересекаются (нет ни одной общей точки).
- `Touch` – если круги касаются друг друга (с точностью $\epsilon = 10^{-5}$).
- `Intersect` – если круги пересекаются

ВХОД	ВЫХОД
0 0 1	Touch
0 2 1	
0 0 1	Intersect
1 1 1	
0 0 3	Do not intersect
5 5 4	
0 0 4	Intersect
5 5 4	
-2 1 4	Touch
2 4 1	

Задача 18. Гамма-функция

Гамма-функция – это обобщение понятия факториала на вещественные числа. Определяется следующим образом:

$$\Gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt$$

Легко вывести, что $\Gamma(n) = (n-1)!$ для натуральных n . Написать функцию, `double gamma(double x)`, которая будет вычислять значение гамма-функции в точке x , при $x > 1$. Для вычисления интеграла использовать метод трапеций с шагом `step = 1e-2`. Суммирование продолжать до тех пор пока площадь трапеции превышает `eps = 1e-10` (то есть 10^{-10}). `step` и `eps` задать как константы. Понадобятся функции `pow` и `exp` из библиотеки `math.h`.

ВХОД	ВЫХОД
2	1.0
6	120.0
20	1.21645e+17
1.5	0.88623
2.5	1.32934
4.14159	7.188082

Задача 19. Размеры типов

Напишите программу, которая будет печатать размеры следующих типов:

- `char`
- `long long`
- `int32_t`
- `double`
- `short`
- `size_t`
- `uint32_t`
- `int[100]`
- `int`
- `int8_t`
- `float`
- `char[100]`

Задача 20. Камень, ножницы, бумага

Представьте, что вы разрабатываете игру "Камень, Ножницы, Бумага". Ваша задача – создать логику игры, используя перечисления и инструкцию `switch`.

- Создайте новый тип перечисления `enum shape` для различных знаков в игре со следующими значениями:
 - ROCK
 - PAPER
 - SCISSORS
- Создайте новый тип перечисления `enum result` (результат игры) со следующими значениями:
 - LOSS
 - DRAW
 - WIN

- Напишите функцию `print_shape`, которая бы принимала на вход объект типа `enum shape` и печатала на экран название передаваемого знака (`"Rock"`, `"Paper"` или `"Scissors"`).
- Напишите функцию `print_result`, которая бы принимала на вход объект типа `enum result` и печатала на экран название передаваемого результата (`"Loss"`, `"Draw"` или `"Win"`).

- Напишите функцию:

```
enum result get_result(enum shape a, enum shape b)
```

которая бы возвращала результат игры с точки зрения первого игрока (игрока `a`).

- Напишите функцию:

```
enum shape get_strength(enum shape s)
```

которая бы принимала на вход знак и возвращала тот знак, который "бьётся" данным знаком. Например, если на вход этой функции придёт `PAPER`, то она должна вернуть `ROCK`.

Необязательные задачи

Задача 1. Треугольник из звёздочек

Напишите функцию `triangle`, которая будет принимать на вход одно целое положительно число `n` и будет печатать на экран прямоугольный треугольник из звёздочек (символов `*`). В функции `main` считайте с экраная число и вызовите функцию `triangle`, передав ей считанное число.

ВХОД	ВЫХОД
5	<pre>*</pre> <pre>**</pre> <pre>***</pre> <pre>****</pre> <pre>*****</pre>
3	<pre>*</pre> <pre>**</pre> <pre>***</pre>
1	<pre>*</pre>

Задача 2. Оставить последнюю цифру

Напишите функцию `last_digits`, которая будет принимать на вход массив целых чисел и количество элементов этого массива. Эта функция должна заменять каждый элемент массива на его последнюю цифру (в десятичной записи числа). Протестируйте эту функцию в функции `main`.

вход функции <code>last_digits</code>	массив после выполнения <code>last_digits</code>
<pre>array: 12 61 426 17 115</pre> <pre>size: 5</pre>	<pre>2 1 6 7 5</pre>
<pre>array: 5 10</pre> <pre>size: 2</pre>	<pre>5 0</pre>

Задача 3. Факториалы

Напишите функцию `factorials`, которая будет принимать на вход массив целых чисел и количество элементов этого массива. Эта функция должна заменять каждый элемент этого массива на его факториал. Напишите вспомогательную функцию `fact`, которая будет принимать одно целое число и возвращать факториал этого числа. Протестируйте эту функцию в функции `main`.

вход функции <code>factorials</code>	массив после выполнения <code>factorials</code>
<pre>array: 3 4 5 6 7</pre> <pre>size: 5</pre>	<pre>6 24 120 720 5040</pre>
<pre>array: 10 11 12</pre> <pre>size: 3</pre>	<pre>3628800 39916800 479001600</pre>

Задача 4. Сортировка

Напишите функцию `sort`, которая будет принимать на вход массив целых чисел и количество элементов этого массива. Эта функция должна сортировать все элементы массива по убыванию. Протестируйте эту функцию в функции `main`.

вход функции <code>sort</code>	массив после выполнения <code>sort</code>
<pre>array: 70 20 80 30 50 40 10 60</pre> <pre>size: 8</pre>	<pre>80 70 60 50 40 30 20 10</pre>
<pre>array: 60 20 80 10</pre> <pre>size: 4</pre>	<pre>80 60 20 10</pre>

Задача 5. Транспонирование

Написать функцию `void transpose(float A[MAX][MAX], int n)`, которая будет принимать на вход матрицу `A` размера `n` на `n` и транспонировать её. Также напишите вспомогательную функцию `print(float A[MAX][MAX], int n)` для печати матрицы на экран. Считайте матрицу из файла `files/matA.txt` с помощью перенаправления потока, транспонируйте её, а результат запишите в новый файл `result.txt`.

Задача 6. Метод Гаусса

Написать программу, которая бы решала линейную систему уравнений $Ax = b$ методом Гаусса.

Главная функция этой программы должна иметь вид:

```
void solve_linear_system(int n, float A[MAX][MAX], float b[], float x[]).
```

Программа должна считывать матрицу A размера n на n и столбец b из файла и записывать результат решения x в новый файл. Предполагайте, что детерминант матрицы не равен нулю. Примерный вид вашей программы:

```
#include <stdio.h>
#define MAX 200

// Возможно понадобится вспомогательная функция для перестановки строк матрицы A
void swap_rows(float A[MAX][MAX], int n, int k, int m)
{
    // Ваш код
}

void solve_linear_system(float A[MAX][MAX], int n, float b[], float x[])
{
    // Ваш код
}

int main()
{
    int n;
    float A[MAX][MAX];
    float b[MAX];
    float x[MAX];

    // Считываем n, A и b
    solve_linear_system(n, A, b, x);
    // Печатаем x
}
```

Проверьте вашу программу на следующих тестах (считывайте из файла методом перенаправления потока):

1. Следующая система:

$$\begin{cases} x_1 + x_2 - x_3 = 9 \\ x_2 + 3x_3 = 3 \\ -x_1 - 2x_3 = 2 \end{cases}$$

Вход для программы должен выглядеть следующим образом:

```
3
1 1 -1
0 1 3
-1 0 -2
9
3
2
```

Решение этой системы: $x = (\frac{2}{3}, 7, -\frac{4}{3}) \approx (0.67, 7.00, -1.33)$

2. Системы из файлов `system1.txt`, `system2.txt`, `system3.txt`. Решения в файлах с именами `x1.txt`, `x2.txt`, `x3.txt` соответственно. Считывайте из файлов методом перенаправления потока.

Задача 7. Объём n -мерного шара

Формула для n -мерного объёма n -мерного шара имеет вид:

$$V_n(R) = \begin{cases} \frac{2(\frac{n-1}{2})! \cdot (4\pi)^{\frac{n-1}{2}}}{n!} R^n, & \text{если } n - \text{нечётное} \\ \frac{\pi^{\frac{n}{2}}}{\frac{n}{2}!} R^n, & \text{если } n - \text{чётное} \end{cases}$$

Напишите программу, которая по заданному n будет вычислять отношение объёма n -мерного куба к объёму вписанному в него n -мерного шара, то есть $\frac{(2R)^n}{V_n(R)}$. Вам может понадобиться функция `row` из библиотеки `math.h`.

ВХОД	ВЫХОД
1	1
2	1.27324
3	1.909859
6	12.384589
10	401.542796
15	85905.301384

Задача 8. Угол

На вход программе поступают компоненты двух векторов. Нужно найти угол между ними в градусах.

ВХОД	ВЫХОД
1 0	90
0 1	
1 0	45
1 1	
-1 0	135
1 1	
-2 8	74.2913
7 4	

Угол α между векторами можно найти из формул для скалярного произведения:

$$\vec{v} \cdot \vec{u} = |\vec{v}| |\vec{u}| \cos(\alpha)$$

$$\vec{v} \cdot \vec{u} = v_x u_x + v_y u_y$$

Вам могут понадобиться следующие функции:

```
double distance(double x1, double y1, double x2, double y2) {
    return sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2));
}
double length(double x, double y) {
    return distance(x, y, 0, 0);
}
double scalar_product(double x1, double y1, double x2, double y2) {
    return x1 * x2 + y1 * y2;
}
const double pi = 3.14159265359;
double to_degrees(double rad) {
    return rad * 180 / pi;
}
```

Задача 9. Бинарный поиск на вещественных числах

Пусть у нас есть монотонно возрастающая функция $f(x)$, а наша задача заключается в том, чтобы найти решение уравнения $f(x) = 0$ на отрезке (l, h) . Причём $f(l) < 0$, а $f(h) > 0$.

Для решения этой задачи можно применить метод бинарного поиска. Для этого находим значение функции в центре отрезка, то есть в точке $m = \frac{l+h}{2}$. Если в этой точке функция положительна или равна нулю, то изменяем значение $h = m$. Если же в этой точке функция отрицательна, то изменяем значение $l = m$. Таким образом отрезок, на котором находится решение был уменьшен в 2 раза. Повторяем эту процедуру до тех пор пока длина отрезка не станет меньше чем $\epsilon = 10^{-10}$.

Напишите программу, которая будет решать эту задачу. Функция $f(x)$ и значения l и h должны задаваться в тексте программы.

$f(x)$, l, h	выход
$f(x) = x^2 - 2$ l = 0, h = 2	1.41421
$f(x) = x^2 - 7$ l = 0, h = 7	2.64575
$f(x) = x^5 + 2x^4 + 5x^2 + 4x - 500$ l = 0, h = 10	3.05614
$f(x) = e^x \ln(x) - 7$ l = 1, h = 5	2.1896095